Phase 2 Report: Data Collection and Preprocessing

Graph Neural Networks for Network Intrusion Detection

Group ID: F26-10

Abdul Rehman (24L-2613) Waniya Sohail (24L-2552) Ghulam Murtaza (24L-2566)

1. Dataset Source and Description

We are using the **CICIDS2017** dataset, published by the Canadian Institute for Cybersecurity at the University of New Brunswick. It is one of the most widely used datasets for intrusion detection research, which means our results can be compared against published work.

Source: https://www.unb.ca/cic/datasets/ids-2017.html

It comes in two versions. We used **GeneratedLabelledFlows** rather than the more common MachineLearningCSV, because the latter has the IP and timestamp columns stripped out. Our project needs IPs to build a graph, so without them there is no graph.

The raw download is 8 CSV files, about 1.12 GB in total, one file per capture session:

File	Date	Rows	Contains
Monday-WorkingHours	3 Jul	529,918	Normal traffic only
Tuesday-WorkingHours	4 Jul	445,909	FTP and SSH brute force
Wednesday-workingHours	5 Jul	692,703	DoS attacks, Heartbleed
Thursday-Morning-WebAttacks	6 Jul	458,968	Web attacks
Thursday-Afternoon-Infiltration	6 Jul	288,602	Infiltration
Friday-Morning	7 Jul	191,033	Botnet
Friday-Afternoon-PortScan	7 Jul	286,467	Port scan
Friday-Afternoon-DDos	7 Jul	225,745	DDoS

Each row is one network flow, meaning one conversation between two machines summarised into numbers. Each file has 85 columns: 7 identifier columns (Flow ID, source and destination IP, source and destination port, protocol, timestamp), 77 numerical traffic features, and 1 label column saying whether that flow was normal or an attack.

2. Data Collection Method

We did not capture this data ourselves. It was collected by the Canadian Institute for Cybersecurity over five working days in July 2017, and we downloaded the published files.

Normal traffic. The researchers profiled 25 real users - how they browse, email, use FTP and SSH - then ran software agents imitating those profiles on a test network to produce realistic background traffic.

Attack traffic. Real attacks were run on a fixed schedule. Monday was left clean as a baseline; Tuesday to Friday each ran a different attack per session. The known schedule is how the flows were labelled.

Turning packets into rows. Traffic was recorded as raw packet captures, then a tool called CICFlowMeter grouped packets into flows and computed each flow's statistics. Our CSVs are that output, so feature extraction was already done for us.

One limitation: the capture covers five weekdays, about 9am to 5pm, on a small test network. No weekends, no night traffic, few machines. Section 4 shows why this matters.

3. Preprocessing Steps

We processed one file at a time, then merged them, which kept memory use low enough for Google Colab. Every step was logged.

3.1 Fixing file problems

- **Encoding.** The files are not UTF-8 and crash a normal read. We had to read them as latin1.
- **Column names.** Every column name except the first has a leading space, for example ' Flow Duration'. We stripped these.
- **Empty rows.** The Thursday morning file has 288,602 completely blank rows at the end. Removing them left exactly 170,366 real rows, which matches the published count for that file.
- **Duplicate column.** 'Fwd Header Length' appears twice. We dropped the copy.

3.2 Fixing the timestamps

This was the least obvious problem. The timestamps are written on a 12-hour clock but the AM/PM marker is missing. An afternoon flow at 3:24pm is stored as '03:24', which reads as 3:24am.

We confirmed this by listing the hours present in all 8 files. Only hours 1 to 12 ever appear, and hours 0, 6, 7 and 13 to 23 never appear. A real 24-hour clock would not produce that gap. Since the captures ran from about 08:55 to 17:05, hours 8 to 12 are morning and hours 1 to 7 are afternoon. We added 12 hours to the afternoon rows. After the fix, each day runs from roughly 08:55 to 17:05 as expected.

Left unfixed, every afternoon flow would sit twelve hours early, and both the time-based split and the Phase 3 time-window graphs would be wrong.

3.3 Cleaning the values

- **Infinite values.** 'Flow Bytes/s' and 'Flow Packets/s' are calculated by dividing by the flow duration, so any flow with zero duration produces infinity. We converted these to missing values.
- **Missing values.** We dropped rows with missing features. There were 2,867 in total, about 0.1 percent, and they are all zero-duration flows that carry no usable timing information. Filling them in would mean inventing data.
- **Constant columns.** Eight columns have the same value in every single row, such as 'Bwd PSH Flags' and 'Fwd Avg Bytes/Bulk'. They carry no information and were removed. This had to be checked on the merged data, not per file, because a column can look constant on one day and vary on another.

3.4 Removing duplicates, and a mistake we caught

Our first attempt compared rows using only the 77 traffic features and the label, treating the ports as identifiers rather than real values. That removed 58 percent of the port scan file and left only 1,956 PortScan flows out of about 158,000.

The cause: a port scan is one machine hitting many ports on one target. Those flows share the same IPs, the same tiny packet counts and the same near-zero duration. The *only* difference is the port. Leaving ports out made thousands of distinct scan attempts look identical.

We corrected this by comparing every column except Flow ID and Timestamp, so ports are included. After the fix, PortScan came back to 158,804 and duplicate removal dropped from about 500,000 rows to roughly 11,000. The class counts now match the published CICIDS2017 figures almost exactly.

3.5 Cleaning log

Day	Loaded	Empty rows	Missing	Duplicates	Final
Monday	529,918	-	437	800	528,681
Tuesday	445,909	-	264	834	444,811
Wednesday	692,703	-	1,297	8,484	682,922
Thursday (web)	458,968	288,602	135	306	169,925
Thursday (infil.)	288,602	-	207	463	287,932

Friday (morning)	191,033	-	122	345	190,566
Friday (portscan)	286,467	-	371	166	285,930
Friday (DDoS)	225,745	-	34	83	225,628
After merge	**2,816,395**			**1,823**	**2,814,572**

The last row is the cross-file check: 1,823 flows appeared in more than one file and were removed once.

3.6 What we deliberately did not do

- **No scaling yet.** The scaler must be fitted on the training data only. Fitting it on everything would leak information from the test set into training. This happens after the split, in Phase 3.
- **No class balancing.** No SMOTE, no oversampling. Detecting rare attacks is exactly what our project is testing, so artificially fixing the imbalance now would remove the thing we are trying to measure.
- **IP addresses kept, but not as features.** We need them to build the graph. They will never be given to a model as input. Section 4 explains why this matters so much for this dataset.

4. Exploratory Analysis

4.1 The classes are extremely imbalanced

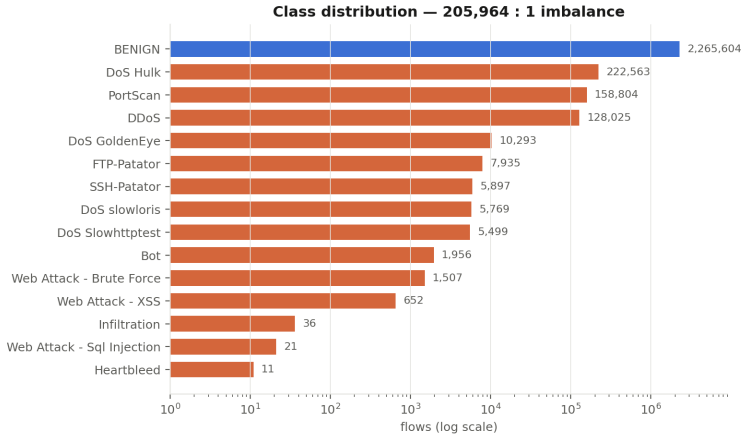


Figure 1: Number of flows per class. The scale is logarithmic, otherwise the rare classes are invisible.

Normal traffic is 80.5 percent of the data, and the largest class is 205,964 times bigger than the smallest. Heartbleed has 11 flows, SQL Injection 21 and Infiltration 36, out of 2.8 million rows. So accuracy is useless here: labelling everything normal scores 80.5 percent while catching nothing. We will use macro F1 and per-class recall, which weigh a rare class as heavily as a common one.

4.2 Attacks appear as short bursts

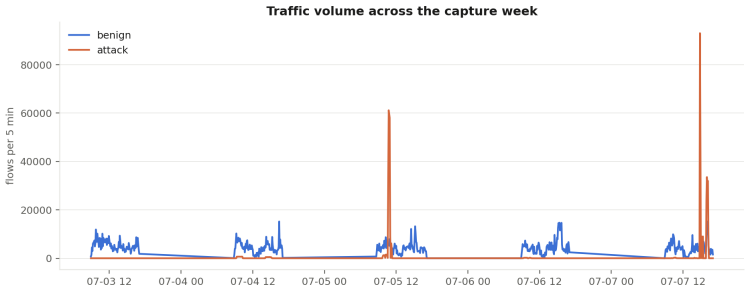


Figure 2: Traffic volume in 5-minute buckets across the week.

Normal traffic follows an office pattern, rising in the morning and falling in the evening, with flat gaps overnight where nothing was captured. The attacks are the tall orange spikes: DoS on Wednesday afternoon, and PortScan and DDoS on Friday afternoon. During those spikes the attack traffic is several times larger than all the normal traffic combined.

The plot also confirms the timestamp fix worked: each day sits inside working hours.

4.3 Many features measure the same thing

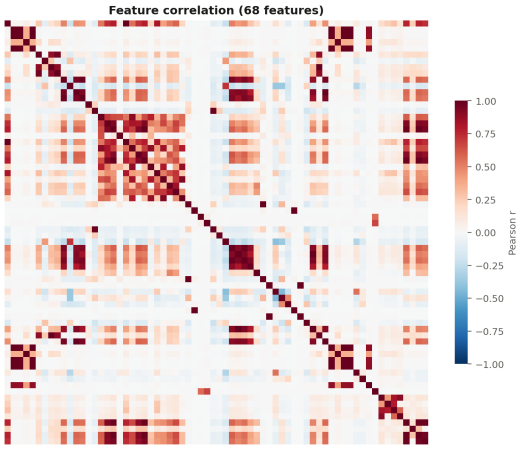


Figure 3: Correlation between the 68 remaining features. Red is positive, blue is negative.

The dark red blocks along the diagonal are groups of features that move together almost perfectly. This is expected, because many of the 77 features are different ways of describing the same packets, for example total bytes, mean packet size and maximum packet size.

We have not removed them. Random Forest handles correlated inputs fine, and cutting features now would make the model comparison less clean.

4.4 The features already separate attacks fairly well

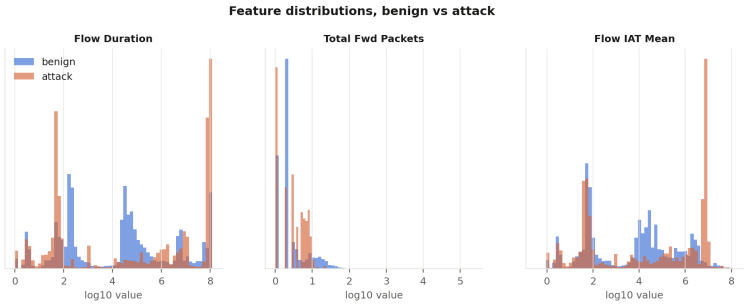


Figure 4: Distribution of three features, normal versus attack, on a log scale.

Attack flows cluster at very short durations and small packet counts, while normal flows spread more widely. The groups overlap but differ clearly.

This is also a warning. If the plain features separate the classes this well, a Random Forest will score very high and leave little room for a graph model to improve on - which is why our hypothesis targets rare and unseen attacks rather than overall accuracy.

4.5 Graph structure, and an important limitation

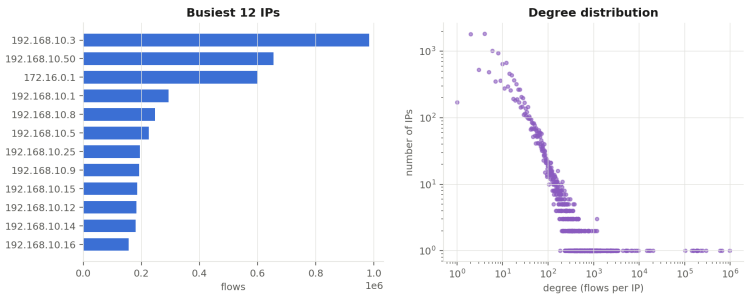


Figure 5: Busiest machines, and how many flows each machine has.

The network has 19,129 unique IPs and 2,814,572 flows between them. The average machine has 294 flows but the median has only 14, so a few machines carry nearly all the traffic - the kind of uneven structure a graph model can use.

However, we also checked where the attacks actually sit in the graph, and the answer limits what we can expect:

Attack	Flows	Source IPs	Target IPs	Distinct connections
PortScan	158,804	1	1	1
DoS Hulk	222,563	1	1	1
DDoS	128,025	2	2	2
Bot	1,956	8	8	12

All 158,804 port scan flows run between a single pair of machines, 172.16.0.1 and 192.168.10.50. The scan spreads across 1,000 different ports, not across different machines. The same is true for most other attacks.

Two things follow. With machines as nodes, the attacks have almost no shape to learn from, since they sit on one or two connections. And the attacking machine 172.16.0.1 is the third busiest address of the week, so any model allowed to see IPs would memorise it and report a meaningless near-perfect score. That is why IPs stay out of the feature set.

For Phase 3 we will therefore build the graph using machine and port together as a node, instead of machine alone. Under that definition the port scan becomes a genuine fan: one attacking endpoint connecting to 1,000 target endpoints. We will verify this before building the models.

5. Final Dataset Summary

Property	Value
Rows (flows)	2,814,572
Features	68 numerical
Identifier columns kept	7 (IPs, ports, protocol, timestamp, flow ID)
Classes	15 (1 normal, 14 attack types)
Normal traffic	2,265,604 (80.5%)
Attack traffic	548,968 (19.5%)
Imbalance ratio	205,964 : 1
Unique machines (nodes)	19,129
Rows removed in cleaning	15,886 (0.56%)
File size	261 MB (Parquet)

Train and test split. We split by time rather than randomly. Monday to Wednesday is training (1,656,414 flows) and Thursday to Friday is testing (1,158,158 flows). A random split would place near-identical flows on both sides and give misleadingly high scores.

One consequence: Bot, PortScan, Infiltration and the Web Attacks occur only on Thursday and Friday, so they land in test and never in training. That follows from the capture schedule, and it suits our research question - it tests whether a model can recognise attacks it has never seen.

Files produced. cicids2017_clean.parquet (the cleaned dataset), cleaning_log.csv (row counts for every step), feature_columns.csv (the 68 features), and figures/ (the five figures above).

Ready for Phase 3. The data is cleaned, timestamps corrected, the split fixed and the graph structure checked. What remains is scaling on the training data, building the endpoint-level graphs, and training.